Funções no PHP

Definição de função

- Trechos de código que podem executar qualquer tipo de tarefa
  - Somar dois números, verificar se é um CPF está correto ou se um valor de uma variável é válido, transformar letras maiúsculas em minúsculas...
- Podem ser usadas em qualquer momento da execução
- Deixam o código dos programas mais organizados e modulares

Função – Sintaxe

Funtion *nome_função* (arg1, arg2 ..., argn)
{

comandos

[**return** <expressão>]

}

- *nome_função* deve ser um nome único, um identificador

- Não se pode iniciar o identificador com números e nem usar caracteres especais como ponto, espaço etc. Entretanto o "_" (sublinhado) pode e é muito usado para separar palavras

- Quando chamada uma função pode receber vários valores

Definição de função

- Quando o PHP encontra uma chamada de função, o fluxo de execução é direcionado para o início do código da função e depois do seu termino o fluxo retorna para aonde foi parado anteriormente e segue o próximo comando

- Na sintaxe o comando *return* também pode ser usado. Sendo
  usado, faz com que a função retorne um valor no local em que foi chamada. O *return* pode ser usado para:
  - Atribuir um valor retornado a uma variável
  - Testar (comandos condicionais) o valor retornado de uma função

- Um função também pode não retornar nenhum valor. Como

Criação de uma função

- Criando uma função e incluindo uma chamada no programa principal

```
function soma_valores($valor1, $valor2, $valor3)
        {
                $soma = $valor1 + $valor2 + $valor3;

                echo "A soma dos valores $valor1, $valor2 e $valor3 ";
                echo "é $soma";
        }

        $n1 = 10;
        $n2 = 20;
        $n3 = 30;
```

Criação de uma função

- As variáveis passadas como parâmetros não precisam ter o mesmo nome dos argumentos definidos na função. Cada um deles assumirá os valores respectivamente dos parâmetros ($valor1 receberá $n1, $valor2 $n2...)

Comando *return* nas funções

- Retorna para ponto de chamada da função o valor que aparece logo depois desse comando. Assim podemos atribuir uma variável (com o operador =) com o valor retornado

- O PHP possui a função *strtoupper* para transformar todas letras de uma string em maiúsculas. Porém, de acordo a versão, esse comando não funciona com acentos. Pode ser criado uma função para resolver isso

Comando *return* nas funções

```php
function maisculo($string){
        $string = strtoupper ($string);
        $string = str_replace ("á","Á",$string);
        $string = str_replace ("é","É",$string);
        $string = str_replace ("í","Í",$string);
        $string = str_replace ("ó","Ó",$string);
        $string = str_replace ("ú","Ú",$string);
        $string = str_replace ("â","Â",$string);
        $string = str_replace ("ê","Ê",$string);
        $string = str_replace ("ô","Ô",$string);
        $string = str_replace ("î","Î",$string);
        $string = str_replace ("Û","U",$string);
        $string = str_replace ("ã","Ã",$string);
        $string = str_replace ("õ","Õ",$string);
        $string = str_replace ("ç","Ç",$string);
        $string = str_replace ("á","Á",$string);
        $string = str_replace ("à","À",$string);

        return $string;
}

$nome = "José Antônio";
$nome_m = maisculo ($nome);
```

- Usamos o função *strtoupper* para deixar os caracteres suportados em maiúsculo. Depois o *str_replace*, que substitui todos os caracteres de um tipo de por outro em uma string, para os acentuados;
- Se não fosse usado o comando *return* todas as alterações dentro da função seriam perdidas
- Existem diversas funções que trabalham com string, basta consultar o manual do PHP

Comando *return* nas funções

- O retorno de uma função pode ser impresso diretamente, sem ser preciso armazená-lo em uma variável

```
function triplo($numero){

        $x = $numero * 3;
        return $x;


}

$valor = 5;

echo "O triplo de $valor é " . triplo($valor);
```

Comando *return* nas funções

```
function capitais(){
        $capitais [] = "Natal";
        $capitais [] = "Belo Horizonte";
        $capitais [] = "Rio de Janeiro";
        $capitais [] = "Goiânia";
        $capitais [] = "Brasília";
        return $capitais;
}
//início do programa principal
$nomes = capitais();
for($i=0; $i<sizeof($nomes); $i++){
        echo "\$nome[$i] vale $nome[$i] <br>";
}
```

- A função capitais não recebe nenhum argumento. Apenas

Passagem de parâmetros: valor e referência

- Quando passamos uma variável como parâmetro em uma função, todas as alterações nelas que ocorrerem dentro da função não serão refletidas fora da função. Isso é chamado de processo de passagem de parâmetros por valor, já que apenas o valor é passado para uso na função

- Quando queremos que variável usada no argumento sofra as alterações internas da função usamos a passagem de parâmetros por referência

Passagem de parâmetros: valor e referência

```
function dobro($valor)
  {
        $valor = 2 * $valor;
  }
 function duplica(&$valor)
  {
        $valor = 2 * $valor;
  }

  $valor = 5;
  dobro($valor);
 echo $valor . "<br>";
  duplica ($valor);
  echo $valor;
```

- Para usar o parâmetro por referência, bastar usar o símbolo & antes da variável
- Enquanto em *dobro*, foi passado somente o valor da variável, em *duplica* o PHP simplesmente criou uma referência para a variável

Passagem de parâmetros: valor e referência

```
function capital($capital, $estado = " Um estado brasileiro")
{
        echo "$estado, capital:   $capital<br>";
}
capital("Natal", "RN");
capital("Belo Horizonte", "MG");
capital("Goiânia");
```

- Também é possível usar valores-padrão nos parâmetros. Caso não seja colocado nenhum argumento, o valor-padrão será assumido, como na última linha do exemplo

- Os valores-padrão devem ser **sempre** os últimos para que o PHP interprete primeiro os parâmetros sem valor-padrão e

Reutilização de Funções

- Imagine que um site possua 50 páginas com a mesma função em cada uma delas. Não é necessário repetir a função em cada uma delas. Caso isso ocorresse, seriam necessárias 50 funções diferentes e uma alteração teria que ser feita em cada uma delas

- Um exemplo simples é o formato de exibição de uma data. O ideal é mudar somente um arquivo e refletir em todas locais que se usa a função. Isso é possível com o uso de *includes*, que será visto mais a frente



Funções Essenciais

Tratamento de Strings, Números e Dados

Tratamento de Strings

Manipulação e limpeza de texto

Extração e Substituição

strlen()

Conta total de caracteres.

```
strlen("PHP") // 3
```

str_replace()

Substitui trechos específicos.

```
str_replace(",", ".", "1,5")
```

substr()

Extrai parte da string.

```
substr("2026", 0, 2) // 20
```

Limpeza e Divisão

`trim()`

Remove espaços das pontas.

```
trim(" admin ") // "admin"
```

`explode()`

Divide string em array.

```
explode("/", "10/05") // ['10', '05']
```

Tratamento de Números

Arredondamento e formatação matemática

Formatação e Arredondamento

`number_format()`

Formata decimais e milhares.

```
number_format(1500.5, 2)
```

`round()`

Arredonda valor flutuante.

```
round(5.6) // 6
```

`ceil() / floor()`

Arredonda para cima/baixo.

```
ceil(5.1) // 6
```

Geração Aleatória



rand($min, $max)

Gera número inteiro pseudoaleatório.

```
rand(1, 100)
```

Manipulação de Dados

Validação de variáveis e arrays

Validação de Variáveis

isset()

Verifica se está definida.

```
isset($_POST['id'])
```

empty()

Checa se valor é vazio.

```
empty($nome)
```

is_numeric()

Valida formato numérico.

```
is_numeric("123.45")
```

Estruturas e Arrays

`implode()`

Junta array em string.

```
implode("-", [1, 2])
```

`array_key_exists()`

Checa chave associativa.

```
array_key_exists("id", $arr)
```

Referências

- PHP Manual:
  - http://www.php.net/manual/pt_BR/index.php